SELL STOCKS DOMAIN KATA · SELF-PACED TRACK

Exercise 2 — Build it from specifications

A self-paced companion for building from durable specifications

> *Companion handout for working on your own. Same domain as Exercise 1 — selling stock from a portfolio — but this time nothing is left to a conversation. Everything the program must do is written down, before any code exists.*

What you are building

The same small piece of financial software as before: an investor sells shares, and the system reports proceeds, profit, and an updated cash balance. This time you're not told a business story and left to guess the gaps — you're handed two files that already settle every behaviour and every class. Your job is to build exactly what they say, then check your result against them.

The specifications

Place both files in `docs/spec/` inside an empty project folder. That is your whole input — nothing else is needed.

- **`sell-stocks-spec.md`** — *behaviour*: preconditions, the FIFO lot-consumption rule, the money definitions, and 24 acceptance criteria (AC-01 … AC-24), each with a starting context, an action, and an exact expected result.
- **`domain-class-diagram.puml`** — *structure*: classes, fields, method signatures, visibility, and relationships (`Portfolio` → `Holding` → `Lot`, plus the value objects).

If the two ever disagree, structure follows the diagram and behaviour follows the spec — and your assistant should tell you about the conflict rather than pick silently, the way it did in Exercise 1.

Pick a language

Java and Python both work; use whichever you prefer.

Java

- Java 21, Maven, a standalone project with its own `pom.xml`, no parent.
- JUnit 5 only, in test scope. No Spring, no persistence, no REST layer.
- Package root: `com.neueda.portfolio.domain`.

- Every monetary amount as `BigDecimal`, scale 2, `HALF_UP` — never `double` or `float`.

Python

- Python 3.11+, a standalone project, `pytest` only — no framework, no persistence layer.
- Package root: `portfolio.domain` (or your project's equivalent).
- Every monetary amount as `decimal.Decimal`, quantized to 2 places with `ROUND_HALF_UP` — never a plain `float`.

The suggested prompt

SUGGESTED PROMPT

```
Implement the domain model described by the two specifications in this
project:

  docs/spec/sell-stocks-spec.md       the behaviour: preconditions, the FIFO
                                       rule, the money definitions, and the
                                       acceptance criteria AC-01 to AC-24
  docs/spec/domain-class-diagram.puml the structure: classes, fields, methods,
                                       visibility and relationships

The specifications are authoritative. Implement what they say and nothing
more: do not add members that are absent from the class diagram, and do not
invent behaviour the acceptance criteria do not describe. If the two ever
disagree, follow the diagram for structure and the specification for
behaviour, and tell me about the conflict rather than choosing silently.

[ Paste in the technical block for your chosen language here. ]

Tests: one test per acceptance criterion, named so the criterion it covers is
obvious. Use the exact numbers from the specification. Compare monetary
amounts with an exact comparison so that 1200 and 1200.00 count as equal.
Assert state, not only return values — the remaining lots, the cash balance,
and the fact that a rejected sale changes nothing at all.

Done when the test suite runs green and every acceptance criterion is covered.
```

Checking what comes back

A green build is not proof — check these, in this order:

1. **The tests run green** — 36 tests from 24 criteria (two are checked against a list of inputs each, one has an extra converse case).
2. **Every criterion has a test.** Walk AC-01 to AC-24. A missing criterion won't show up as a failure.

3. **The numbers are right.** Selling 12 of the baseline holding must give proceeds 1800.00, cost basis 1240.00, profit 560.00, leaving one lot of 3 shares at 120.00 — not a blended-average cost basis of 1280.00.
4. **A rejected sale changes nothing.** Selling 16 of 15 shares must leave the lots and the cash balance untouched — validation before mutation, not during it.
5. **Nothing extra was invented.** Compare the classes against the diagram: no extra fields, no `Transaction` class, no cached `profit` field — if it's not in the diagram, it shouldn't exist.

Worth trying afterwards

- Delete the class diagram and regenerate from the behaviour spec alone — what does the assistant now decide on its own?
- Change one number in the specification (say, the second lot's price) and regenerate. Everything downstream should follow, because the specification is the source of truth, not your memory of a conversation.

What's next

You've now built the same domain twice: once from a conversation, once from durable specifications that exist before any code does. Exercise 3 asks whether the *working method* itself — planning, implementing, reviewing — can be made just as durable.